JS_DAY_6_HANDS_ON_GOPIKRISHNA

JS_DAY_6_HANDS_ON_PROBLEM_1_GOPIKRISHNA

1.Assessment Problem_1:

Requirements

- A textarea for writing notes.
- A Save button (using onclick).
- A Clear button.

Notes must:

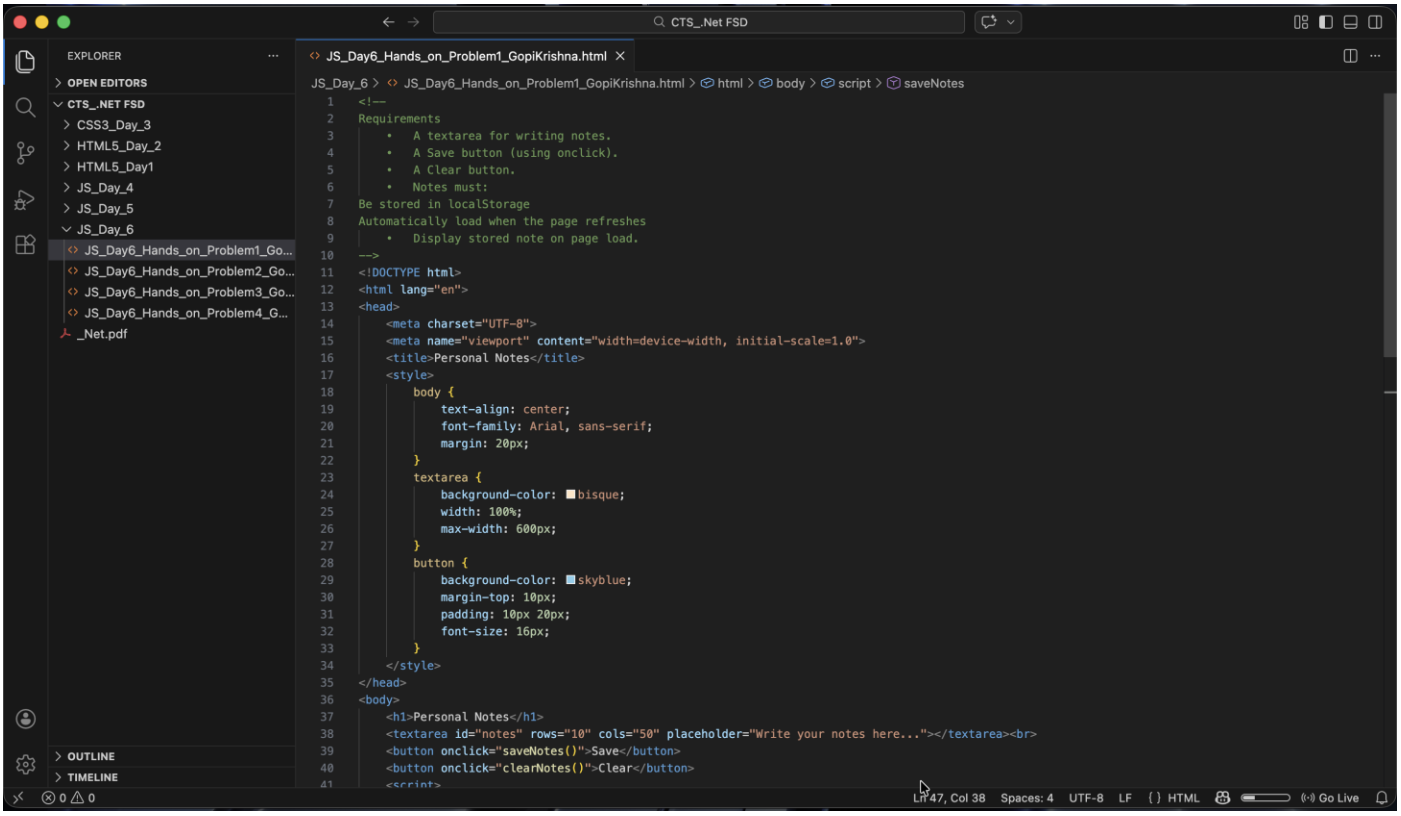
- Be stored in localStorage
- Automatically load when the page refreshes
- Display stored note on page load.

2.Actual Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Personal Notes</title>
   <style>
      body {
         text-align: center;
         font-family: Arial, sans-serif;
         margin: 20px;
      }
      textarea {
         background-color: bisque;
         width: 100%;
         max-width: 600px;
      }
      button {
         background-color: skyblue;
         margin-top: 10px;
         padding: 10px 20px;
         font-size: 16px;
      }
   </style>
</head>
<body>
   <h1>Personal Notes</h1>
   <textarea id="notes" rows="10" cols="50" placeholder="Write your notes here..."></textarea><br>
   <button onclick="saveNotes()">Save</button>
   <button onclick="clearNotes()">Clear</button>
   <script>

      function saveNotes() {
         const notes = document.getElementById('notes').value;
         if (notes.trim() !== "") {
            localStorage.setItem('personalNotes', notes);
            alert('Note saved!');
         } else {
```

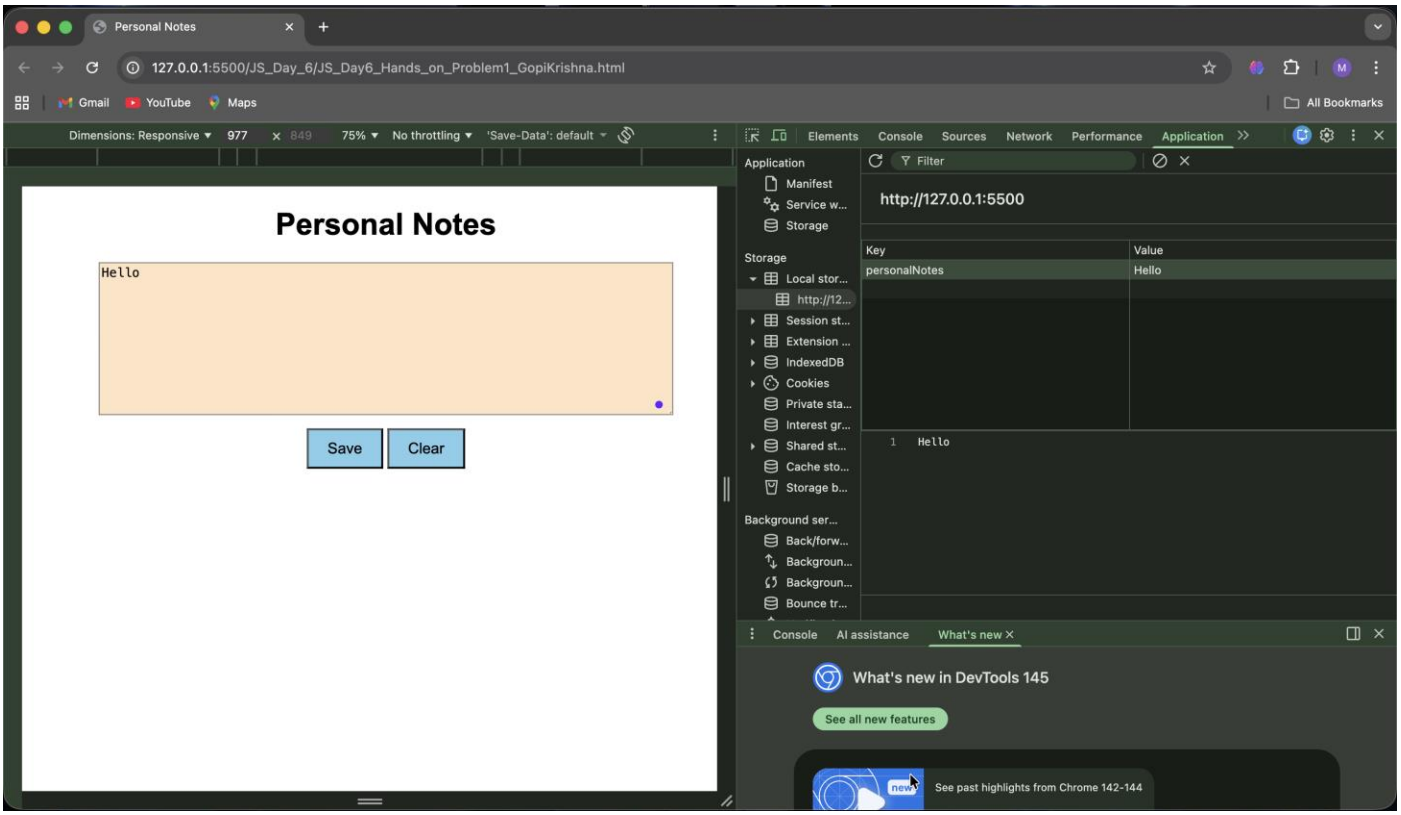
```
            alert('Please  enter some notes before saving.');
        }
    }

    function clearNotes() {
        localStorage.removeItem('personalNotes');
        document.getElementById('notes').value = '';
        alert('Note cleared!');
    }

    window.onload = function() {
        const savedNotes = localStorage.getItem('personalNotes');
        if (savedNotes) {
            document.getElementById('notes').value = savedNotes;
        }
    }
  </script>
</body>
</html>
```

3.Code Screenshot



```html
<html lang="en">
<body>
    <h1>Personal Notes</h1>
    <textarea id="notes" rows="10" cols="50" placeholder="Write your notes here..."></textarea><br>
    <button onclick="saveNotes()">Save</button>
    <button onclick="clearNotes()">Clear</button>
    <script>

        function saveNotes() {
            const notes = document.getElementById('notes').value;
            if (notes.trim() !== "") {
                localStorage.setItem('personalNotes', notes);
                alert('Note saved!');
            } else {
                alert('Please enter some notes before saving.');
            }
        }

        function clearNotes() {
            localStorage.removeItem('personalNotes');
            document.getElementById('notes').value = '';
            alert('Note cleared!');
        }

        window.onload = function() {
            const savedNotes = localStorage.getItem('personalNotes');
            if (savedNotes) {
                document.getElementById('notes').value = savedNotes;
            }
        }
    </script>
</body>
</html>
```

4.Output Screenshot



5.Explanation

This is a client-side notes application that uses the browser's Web Storage API to store user input persistently as a key–value pair in localStorage. Inline onclick handlers trigger save and clear operations, where the note is written using localStorage.setItem() and removed using localStorage.removeItem(). On page load, the window.onload event retrieves the stored value via localStorage.getItem() and repopulates the textarea, ensuring the data persists across page refreshes without any backend involvement.

JS_DAY_6_HANDS_ON_PROBLEM_2_GOPIKRISHNA

1. Assessment Problem_2:

Requirements

- Fields:
  - Name
  - Email
  - Age
- Use:
  - onchange
  - onclick
- Validate:
  - Name cannot be empty
  - Email must contain "@"
  - Age must be greater than 18
- Display validation message dynamically.
- Store valid user data in sessionStorage.

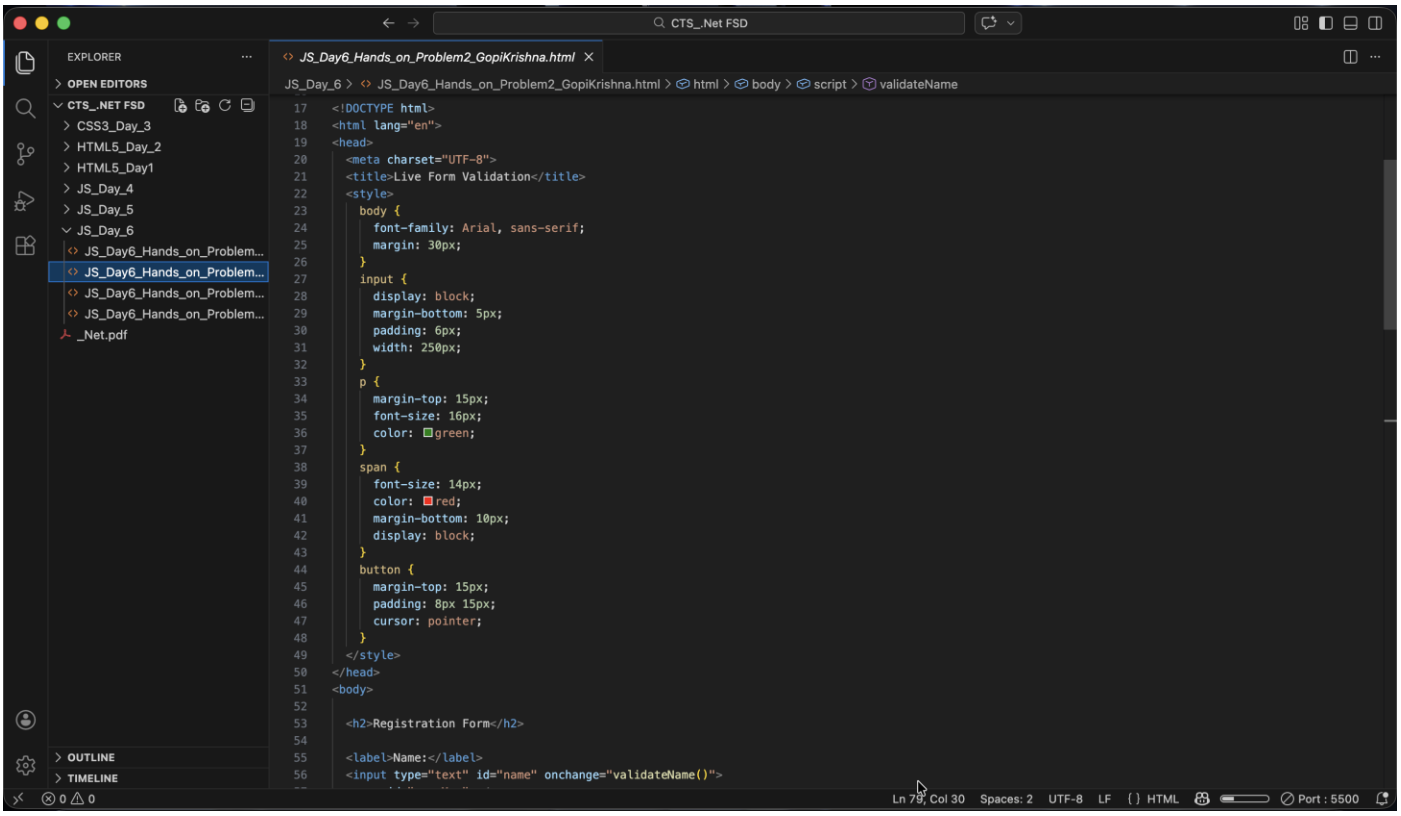
2. Actual Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Live Form Validation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
    }
    input {
      display: block;
      margin-bottom: 5px;
      padding: 6px;
      width: 250px;
    }
    p {
      margin-top: 15px;
      font-size: 16px;
      color: green;
    }
    span {
      font-size: 14px;
      color: red;
      margin-bottom: 10px;
      display: block;
    }
    button {
      margin-top: 15px;
      padding: 8px 15px;
      cursor: pointer;
    }
  </style>
</head>
```

```
<body>

  <h2>Registration Form</h2>

  <label>Name:</label>
  <input type="text" id="name" onchange="validateName()">
  <span id="nameMsg"></span>

  <label>Email:</label>
  <input type="text" id="email" onchange="validateEmail()">
  <span id="emailMsg"></span>

  <label>Age:</label>
  <input type="number" id="age" onchange="validateAge()">
  <span id="ageMsg"></span>

  <button onclick="saveData()">Register</button>
  <p id="finalMsg"></p>

  <script>
    function validateName() {
      let name = document.getElementById("name").value;
      let msg = document.getElementById("nameMsg");

      if (name === "") {
        msg.textContent = "Name cannot be empty";
        return false;
      } else {
        msg.textContent = "";
        return true;
      }
    }

    function validateEmail() {
      let email = document.getElementById("email").value;
      let msg = document.getElementById("emailMsg");

      if (!email.includes("@"))  {
        msg.textContent = "Email must contain '@'";
        return false;
      } else {
        msg.textContent = "";
        return true;
      }
    }

    function validateAge() {
      let age = document.getElementById("age").value;
      let msg = document.getElementById("ageMsg");

      if (age <= 18) {
        msg.textContent = "Age must be greater than 18";
        return false;
      } else {
        msg.textContent = "";
```

```
      return true;
    }
  }

  function saveData() {
    if (validateName() && validateEmail() && validateAge()) {
      sessionStorage.setItem("name", document.getElementById("name").value);
      sessionStorage.setItem("email", document.getElementById("email").value);
      sessionStorage.setItem("age", document.getElementById("age").value);

      document.getElementById("finalMsg").textContent =
        "Registration successful! Data stored in sessionStorage.";
    } else {
      document.getElementById("finalMsg").textContent =
        "";
    }
  }
</script>

</body>
</html>
```

3. Code Screenshots



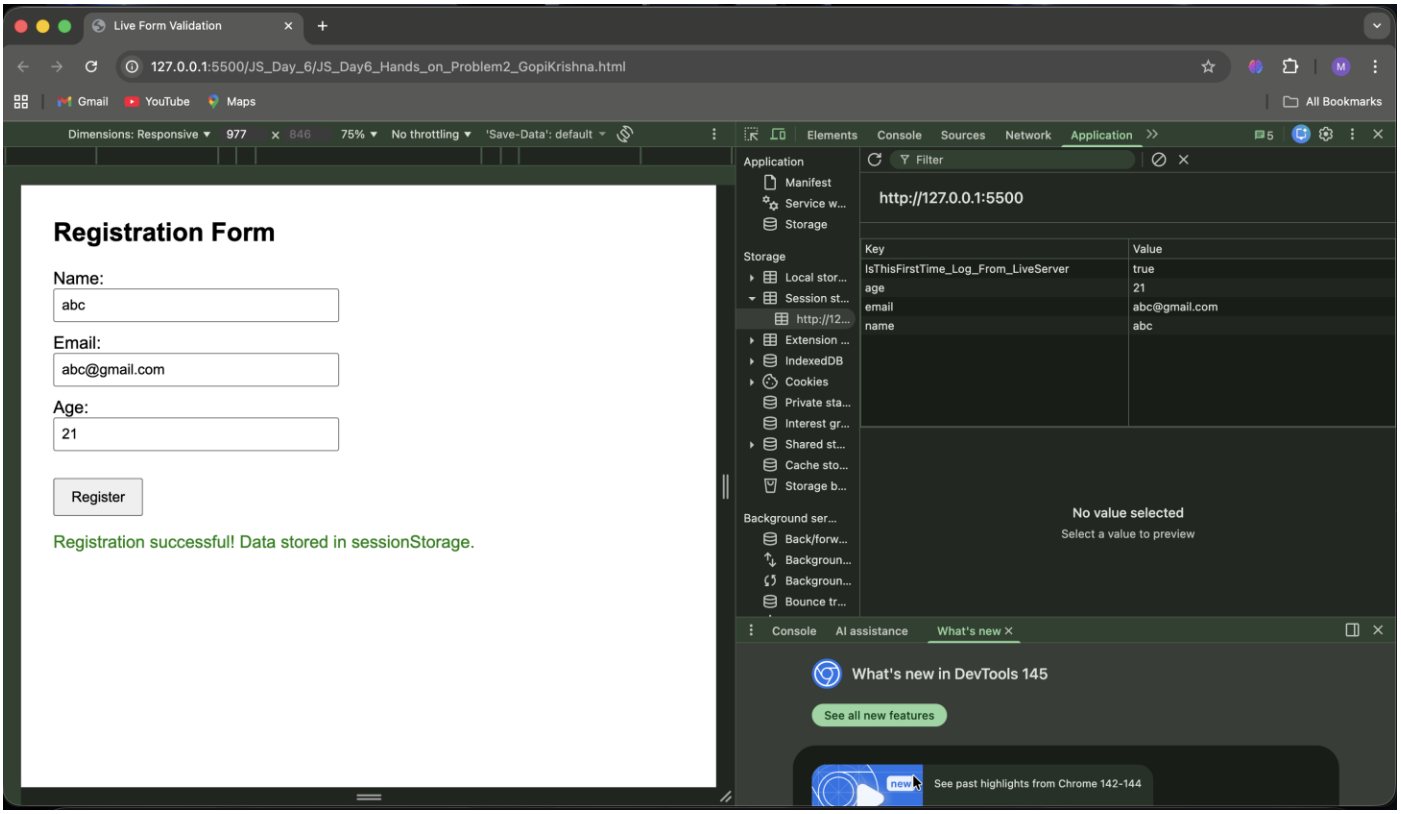
JS_Day6_Hands_on_Problem2_GopiKrishna.html

```html
<html lang="en">
<body>
  <label>Name:</label>
  <input type="text" id="name" onchange="validateName()">
  <span id="nameMsg"></span>

  <label>Email:</label>
  <input type="text" id="email" onchange="validateEmail()">
  <span id="emailMsg"></span>

  <label>Age:</label>
  <input type="number" id="age" onchange="validateAge()">
  <span id="ageMsg"></span>

  <button onclick="saveData()">Register</button>
  <p id="finalMsg"></p>

  <script>
    function validateName() {
      let name = document.getElementById("name").value;
      let msg = document.getElementById("nameMsg");

      if (name === "") {
        msg.textContent = "Name cannot be empty";
        return false;
      } else {
        msg.textContent = "";
        return true;
      }
    }

    function validateEmail() {
      let email = document.getElementById("email").value;
      let msg = document.getElementById("emailMsg");

      if (!email.includes("@")) {
        msg.textContent = "Email must contain '@'";
        return false;
      } else {
        msg.textContent = "";
        return true;
```

JS_Day6_Hands_on_Problem2_GopiKrishna.html

```html
<html lang="en">
<body>
  <script>
    function validateEmail() {
      if (!email.includes("@")) {
        msg.textContent = "Email must contain '@'";
        return false;
      } else {
        msg.textContent = "";
        return true;
      }
    }
    function validateAge() {
      let age = document.getElementById("age").value;
      let msg = document.getElementById("ageMsg");

      if (age <= 18) {
        msg.textContent = "Age must be greater than 18";
        return false;
      } else {
        msg.textContent = "";
        return true;
      }
    }
    function saveData() {
      if (validateName() && validateEmail() && validateAge()) {
        sessionStorage.setItem("name", document.getElementById("name").value);
        sessionStorage.setItem("email", document.getElementById("email").value);
        sessionStorage.setItem("age", document.getElementById("age").value);

        document.getElementById("finalMsg").textContent =
          "Registration successful! Data stored in sessionStorage.";
      } else {
        document.getElementById("finalMsg").textContent =
          "";
      }
    }
  </script>
</body>
</html>
```

4.Output Screenshot



5. Explanation

This application implements client-side live form validation using inline onchange and onclick events to validate user input in real time without any backend. Each input field (name, email, age) triggers a corresponding validation function on value change, where DOM access is used to read input values and dynamically update error messages using textContent. The validation logic enforces mandatory name input, basic email format checking using the presence of "@", and an age constraint greater than 18. On successful validation of all fields, the onclick handler stores the validated data as key–value pairs in sessionStorage using sessionStorage.setItem(), ensuring the data persists only for the current browser session, while providing immediate user feedback through dynamic DOM updates.

JS_DAY_6_HANDS_ON_PROBLEM_3_GOPIKRISHNA

1. Assessment Problem_3:

Requirements:

- Button: **Get My Location**
- Use:
  - navigator.geolocation.getCurrentPosition()
- Display:
  - Latitude
  - Longitude
- Handle:
  - Permission denied
  - Timeout
  - Location unavailable
- Save last 5 location entries in localStorage.
- Display location history on page load.

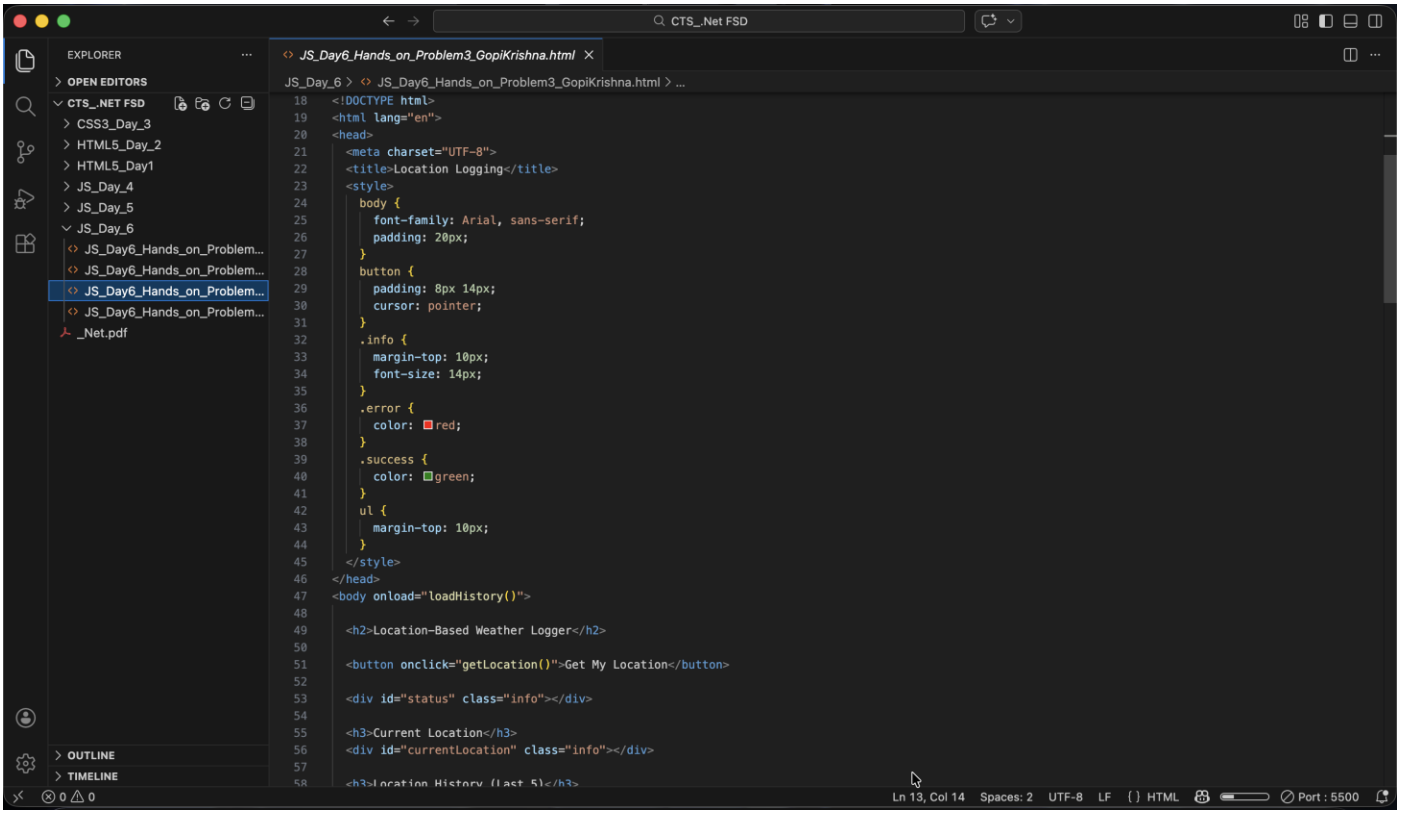
2. Actual Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Location Logging</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    button {
      padding: 8px 14px;
      cursor: pointer;
    }
    .info {
      margin-top: 10px;
      font-size: 14px;
    }
    .error {
      color: red;
    }
    .success {
      color: green;
    }
    ul {
      margin-top: 10px;
    }
  </style>
</head>
<body onload="loadHistory()">

  <h2>Location-Based Weather Logger</h2>

  <button onclick="getLocation()">Get My Location</button>

  <div id="status" class="info"></div>

  <h3>Current Location</h3>
  <div id="currentLocation" class="info"></div>

  <h3>Location History (Last 5)</h3>
  <ul id="historyList"></ul>

  <script>
    function getLocation() {
      let status = document.getElementById("status");

      if (!navigator.geolocation) {
        status.textContent = "Geolocation is not supported by this browser.";
        status.className = "info error";
        return;
      }

      status.textContent = "Fetching location...";
      status.className = "info";
```

```
  navigator.geolocation.getCurrentPosition(
    successCallback,
    errorCallback,
    { timeout: 10000 }
  );
}

function successCallback(position) {
  let lat = position.coords.latitude;
  let lon = position.coords.longitude;

  document.getElementById("currentLocation").textContent =
    "Latitude: " + lat + " | Longitude: " + lon;

  document.getElementById("status").textContent = "Location fetched successfully";
  document.getElementById("status").className = "info success";

  saveLocation(lat, lon);
  loadHistory();
}

function errorCallback(error) {
  let msg = "";

  switch (error.code) {
    case error.PERMISSION_DENIED:
      msg = "Permission denied by user.";
      break;
    case error.POSITION_UNAVAILABLE:
      msg = "Location information unavailable.";
      break;
    case error.TIMEOUT:
      msg = "Location request timed out.";
      break;
    default:
      msg = "Unknown error occurred.";
  }

  let status = document.getElementById("status");
  status.textContent = msg;
  status.className = "info error";
}

function saveLocation(lat, lon) {
  let history = JSON.parse(localStorage.getItem("locations")) || [];

  let entry = {
    latitude: lat,
    longitude: lon,
    time: new Date().toLocaleString()
  };

  history.unshift(entry);
```

```
    if (history.length > 5) {
      history.pop();
    }

    localStorage.setItem("locations", JSON.stringify(history));
  }

  function loadHistory() {
    let history = JSON.parse(localStorage.getItem("locations")) || [];
    let list = document.getElementById("historyList");
    list.innerHTML = "";

    history.forEach(item => {
      let li = document.createElement("li");
      li.textContent =
        item.latitude + ", " + item.longitude + " (" + item.time + ")";
      list.appendChild(li);
    });
  }
</script>

</body>
</html>
```

3. Code Screenshots



JS_Day6_Hands_on_Problem3_GopiKrishna.html

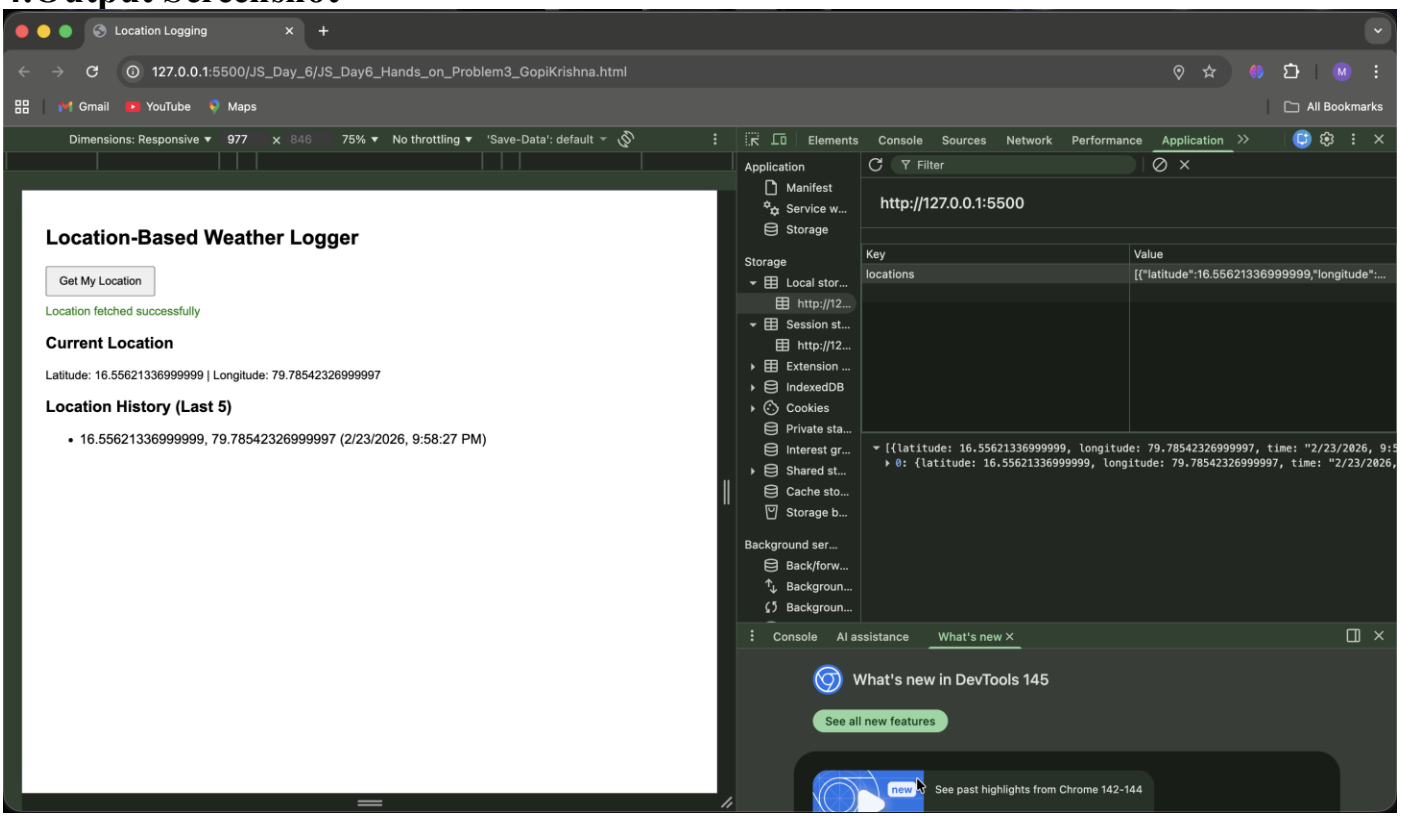
```html
<html lang="en">
<body onload="loadHistory()">
  <h3>Location History (Last 5)</h3>
  <ul id="historyList"></ul>

  <script>
    function getLocation() {
      let status = document.getElementById("status");

      if (!navigator.geolocation) {
        status.textContent = "Geolocation is not supported by this browser.";
        status.className = "info error";
        return;
      }

      status.textContent = "Fetching location...";
      status.className = "info";

      navigator.geolocation.getCurrentPosition(
        successCallback,
        errorCallback,
        { timeout: 10000 }
      );
    }

    function successCallback(position) {
      let lat = position.coords.latitude;
      let lon = position.coords.longitude;

      document.getElementById("currentLocation").textContent =
        "Latitude: " + lat + " | Longitude: " + lon;

      document.getElementById("status").textContent = "Location fetched successfully";
      document.getElementById("status").className = "info success";

      saveLocation(lat, lon);
      loadHistory();
    }

    function errorCallback(error) {
      let msg = "";
```

JS_Day6_Hands_on_Problem3_GopiKrishna.html

```html
<html lang="en">
<body onload="loadHistory()">
  <script>
    function errorCallback(error) {
      let msg = "";

      switch (error.code) {
        case error.PERMISSION_DENIED:
          msg = "Permission denied by user.";
          break;
        case error.POSITION_UNAVAILABLE:
          msg = "Location information unavailable.";
          break;
        case error.TIMEOUT:
          msg = "Location request timed out.";
          break;
        default:
          msg = "Unknown error occurred.";
      }

      let status = document.getElementById("status");
      status.textContent = msg;
      status.className = "info error";
    }

    function saveLocation(lat, lon) {
      let history = JSON.parse(localStorage.getItem("locations")) || [];

      let entry = {
        latitude: lat,
        longitude: lon,
        time: new Date().toLocaleString()
      };

      history.unshift(entry);

      if (history.length > 5) {
        history.pop();
      }
```

```html
<html lang="en">
<body onload="loadHistory()">
  <script>
    function saveLocation(lat, lon) {
      if (history.length > 5) {
        history.pop();
      }

      localStorage.setItem("locations", JSON.stringify(history));
    }

    function loadHistory() {
      let history = JSON.parse(localStorage.getItem("locations")) || [];
      let list = document.getElementById("historyList");
      list.innerHTML = "";

      history.forEach(item => {
        let li = document.createElement("li");
        li.textContent =
          item.latitude + ", " + item.longitude + " (" + item.time + ")";
        list.appendChild(li);
      });
    }
  </script>

</body>
</html>
```

4.Output Screenshot



5. Explanation

This application uses the browser's Geolocation API to fetch the user's latitude and longitude through an inline onclick event and the asynchronous navigator.geolocation.getCurrentPosition() method. On success, the coordinates are displayed dynamically and stored as structured objects in localStorage using JSON.stringify(), while on page load the data is retrieved with JSON.parse() and rendered as a location history limited to the last five entries. The implementation includes explicit success and error callbacks to handle permission denial, timeouts, and unavailable location scenarios, demonstrating client-side permission handling, API error management, and persistent structured storage without any backend.

JS_DAY_6_HANDS_ON_PROBLEM_4_GOPIKRISHNA

1. Assessment Problem_4:

Requirements

- Fields:
  - Expense Title
  - Amount
  - Date
- Buttons:
  - Add Expense
  - View Expenses
  - Delete Expense
- Use:
  - Client-side database (Web SQL or IndexedDB)
- Execute SQL-like queries:
  - CREATE TABLE
  - INSERT
  - SELECT
  - DELETE
- Maintain transaction handling.
- Display expense list dynamically.

2. Actual Code:

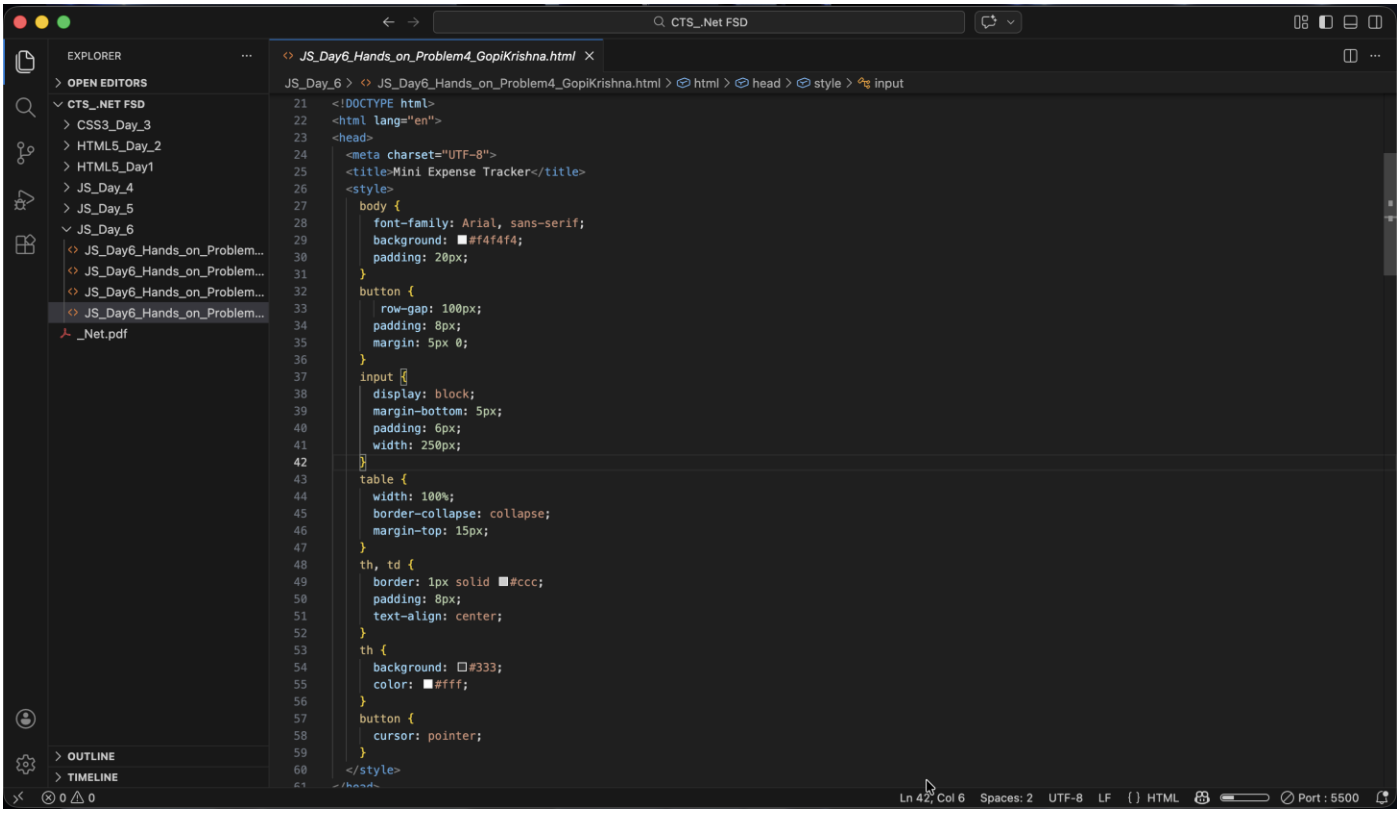
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Mini Expense Tracker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f4f4;
      padding: 20px;
    }
    button {
       row-gap: 100px;
      padding: 8px;
      margin: 5px 0;
    }
    input {
      display: block;
      margin-bottom: 5px;
      padding: 6px;
      width: 250px;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 15px;
    }
    th, td {
      border: 1px solid #ccc;
      padding: 8px;
      text-align: center;
    }
    th {
```

```
      background: #333;
      color: #fff;
    }
    button {
      cursor: pointer;
    }
  </style>
</head>
<body>

    <h2>Mini Expense Tracker (IndexedDB)</h2>

    <label>Expense Title:</label>
    <input type="text" id="title" placeholder="Expense Title"><br>

    <label>Amount:</label>
    <input type="number" id="amount" placeholder="Amount"><br>

    <label>Date:</label>
    <input type="date" id="date"><br>

<button onclick="addExpense()">Add Expense</button>
<button onclick="viewExpenses()">View Expenses</button>

<table>
  <thead>
    <tr>
      <th>Title</th>
      <th>Amount</th>
      <th>Date</th>
      <th>Delete</th>
    </tr>
  </thead>
  <tbody id="expenseList"></tbody>
</table>

<script>
  let db;

  const request = indexedDB.open("ExpenseDB", 1);

  request.onupgradeneeded = function (event) {
    db = event.target.result;

    const store = db.createObjectStore("expenses", {
      keyPath: "id",
      autoIncrement: true
    });

    console.log("Object Store (Table) Created");
  };

  request.onsuccess = function (event) {
    db = event.target.result;
    console.log("Database Connected");
```

```
};

request.onerror = function () {
  console.error("Database Error");
};

function addExpense() {
  const title = document.getElementById("title").value;
  const amount = document.getElementById("amount").value;
  const date = document.getElementById("date").value;

  if (!title || !amount || !date) {
    alert("All fields are required");
    return;
  }

  const transaction = db.transaction("expenses", "readwrite");
  const store = transaction.objectStore("expenses");

  const expense = { title, amount, date };
  const request = store.add(expense);

  request.onsuccess = () => {
    alert("Expense Added");
    viewExpenses();
  };

  request.onerror = () => {
    console.error("Insert Error");
  };

  transaction.onerror = () => {
    console.error("Transaction Failed");
  };
}

function viewExpenses() {
  const transaction = db.transaction("expenses", "readonly");
  const store = transaction.objectStore("expenses");
  const request = store.getAll();

  request.onsuccess = function () {
    const list = document.getElementById("expenseList");
    list.innerHTML = "";

    request.result.forEach(exp => {
      list.innerHTML += `
        <tr>
          <td>${exp.title}</td>
          <td>${exp.amount}</td>
          <td>${exp.date}</td>
          <td>
            <button onclick="deleteExpense(${exp.id})">Delete</button>
          </td>
        </tr>
```

```
      `;
    });
  };

  request.onerror = () => {
    console.error("Select  Query Failed");
  };
}

function deleteExpense(id) {
  const transaction = db.transaction("expenses", "readwrite");
  const store = transaction.objectStore("expenses");
  const request = store.delete(id);

  request.onsuccess = () => {
    alert("Expense  Deleted");
    viewExpenses();
  };

  request.onerror = () => {
    console.error("Delete  Failed");
  };
}
</script>

</body>
</html>
```

3. Code Screenshots



JS_Day6_Hands_on_Problem4_GopiKrishna.html

JS_Day_6 > JS_Day6_Hands_on_Problem4_GopiKrishna.html > html > head > style > input

```html
<html lang="en">
<body>
    <h2>Mini Expense Tracker (IndexedDB)</h2>

    <label>Expense Title:</label>
    <input type="text" id="title" placeholder="Expense Title"><br>

    <label>Amount:</label>
    <input type="number" id="amount" placeholder="Amount"><br>

    <label>Date:</label>
    <input type="date" id="date"><br>

<button onclick="addExpense()">Add Expense</button>
<button onclick="viewExpenses()">View Expenses</button>

<table>
  <thead>
    <tr>
      <th>Title</th>
      <th>Amount</th>
      <th>Date</th>
      <th>Delete</th>
    </tr>
  </thead>
  <tbody id="expenseList"></tbody>
</table>

<script>
  let db;

  const request = indexedDB.open("ExpenseDB", 1);

  request.onupgradeneeded = function (event) {
    db = event.target.result;

    const store = db.createObjectStore("expenses", {
      keyPath: "id",
      autoIncrement: true
    });
```

JS_Day6_Hands_on_Problem4_GopiKrishna.html

JS_Day_6 > JS_Day6_Hands_on_Problem4_GopiKrishna.html > html > head > style > input

```html
<html lang="en">
<body>
<script>
  request.onupgradeneeded = function (event) {
    console.log("Object Store (Table) Created");
  };

  request.onsuccess = function (event) {
    db = event.target.result;
    console.log("Database Connected");
  };

  request.onerror = function () {
    console.error("Database Error");
  };

  function addExpense() {
    const title = document.getElementById("title").value;
    const amount = document.getElementById("amount").value;
    const date = document.getElementById("date").value;

    if (!title || !amount || !date) {
      alert("All fields are required");
      return;
    }

    const transaction = db.transaction("expenses", "readwrite");
    const store = transaction.objectStore("expenses");

    const expense = { title, amount, date };
    const request = store.add(expense);

    request.onsuccess = () => {
      alert("Expense Added");
      viewExpenses();
    };

    request.onerror = () => {
      console.error("Insert Error");
    };
```

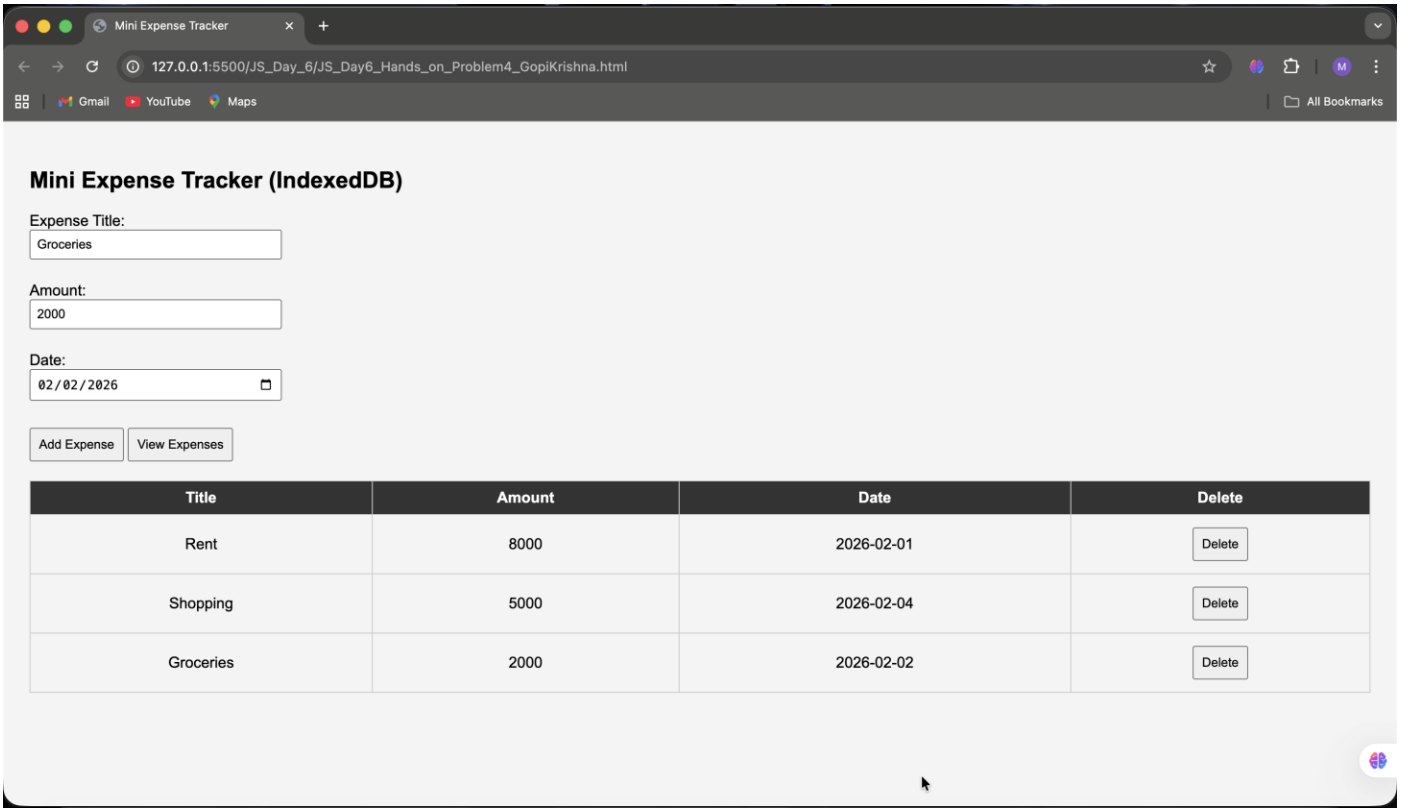
JS_Day6_Hands_on_Problem4_GopiKrishna.html

```html
<html lang="en">
<body>
<script>
  function addExpense() {
    transaction.onerror = () => {
      console.error("Transaction Failed");
    };
  }

  function viewExpenses() {
    const transaction = db.transaction("expenses", "readonly");
    const store = transaction.objectStore("expenses");
    const request = store.getAll();

    request.onsuccess = function () {
      const list = document.getElementById("expenseList");
      list.innerHTML = "";

      request.result.forEach(exp => {
        list.innerHTML += `
          <tr>
            <td>${exp.title}</td>
            <td>${exp.amount}</td>
            <td>${exp.date}</td>
            <td>
              <button onclick="deleteExpense(${exp.id})">Delete</button>
            </td>
          </tr>
        `;
      });
    };

    request.onerror = () => {
      console.error("Select Query Failed");
    };
  }

  function deleteExpense(id) {
    const transaction = db.transaction("expenses", "readwrite");
    const store = transaction.objectStore("expenses");
```

JS_Day6_Hands_on_Problem4_GopiKrishna.html

```html
<html lang="en">
<body>
<script>
  function deleteExpense(id) {
    const transaction = db.transaction("expenses", "readwrite");
    const store = transaction.objectStore("expenses");
    const request = store.delete(id);

    request.onsuccess = () => {
      alert("Expense Deleted");
      viewExpenses();
    };

    request.onerror = () => {
      console.error("Delete Failed");
    };
  }
</script>

</body>
</html>
```

4.Output Screenshot



5. Explanation

This mini expense tracker uses IndexedDB as a client-side database to persist expense records without any backend. An objectStore is created to simulate a table, and all CRUD operations are executed within explicit transactions, where add() performs INSERT, getAll() performs SELECT, and delete() performs DELETE operations. Data is rendered dynamically in the DOM after each transaction, with proper handling of transaction and query errors, demonstrating SQL-like data management, transactional control, and persistent structured storage entirely in the browser using pure HTML and JavaScript.